

Nhập xuất & Biểu thức (I/O & Expressions)

Nội dung

1. Lập trình là gì? Chương trình đầu tiên
2. Biến và kiểu dữ liệu
3. Nhập dữ liệu (Input)
4. Xuất dữ liệu (Output)
5. Các phép toán số học
6. Ví dụ hoàn chỉnh: Bài toán "Tổng hai số"
7. Xuất số thực với độ chính xác
8. Ví dụ: Tính diện tích và chu vi hình chữ nhật
9. Thứ tự ưu tiên phép toán
10. Mẹo và lỗi thường gặp
 - Lỗi 1: Tràn số (Overflow) trong C++
 - Lỗi 2: Chia nguyên bất ngờ trong C++
 - Lỗi 3: In thừa hoặc thiếu
 - Lỗi 4: Quên đọc hết input
 - Mẹo chung

1. Lập trình là gì? Chương trình đầu tiên

Lập trình là viết các chỉ thị để máy tính thực hiện theo. Một chương trình đơn giản nhất là in một dòng chữ ra màn hình:

C++

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

Python

```
print("Hello, World!")
```

Giải thích:

- **C++:** `#include <iostream>` cho phép sử dụng `cout` (xuất) và `cin` (nhập). `using namespace std;` giúp viết gọn hơn. Hàm `main()` là điểm bắt đầu chương trình.
- **Python:** Chỉ cần gọi `print()` — đơn giản hơn nhiều!

Trên LQDOJ (và các online judge khác), chương trình của bạn sẽ đọc dữ liệu từ input và in kết quả ra output — không có giao diện đồ họa, chỉ có văn bản (text).

2. Biến và kiểu dữ liệu

Biến (variable) là "hộp chứa" lưu trữ giá trị trong bộ nhớ. Mỗi biến có một kiểu dữ liệu xác định loại giá trị nó chứa.

Kiểu Dữ Liệu	C++	PYTHON	VÍ DỤ
Số nguyên	int, long long	int	42, -7, 1000000
Số thực	float, double	float	3.14, -0.5
Xâu ký tự	string	str	"Hello", "LQDOJ"

Khai báo biến:

C++

```
int a = 10; // Số nguyên
double pi = 3.14159; // Số thực (dùng double cho chính xác)
string name = "LQDOJ"; // Xâu ký tự
long long big = 10000000000000LL; // Số nguyên lớn (tối ~9.2 * 1018)
```

Python

```
a = 10 # Số nguyên (tự động, không giới hạn kích thước!)
pi = 3.14159 # Số thực
name = "LQDOJ" # Xâu ký tự
big = 10000000000000 # Python xử lý số lớn tự động
```

Lưu ý quan trọng về C++:

- `int` chứa được giá trị tối khoảng 2×10^9 (2 tỷ).
- Nếu bài toán có số lớn hơn, dùng `long long` (tối khoảng 9.2×10^{18}).
- Dùng `double` thay vì `float` để có độ chính xác cao hơn với số thực.

3. Nhập dữ liệu (Input)

Trên online judge, chương trình đọc dữ liệu từ standard input (stdin). Mỗi bài toán sẽ mô tả rõ format input.

Ví dụ: Đọc hai số nguyên a và b từ input:

C++

```
#include <iostream>
using namespace std;
int main() {
    int a, b;
    cin >> a >> b;
    // Bây giờ a và b đã chứa giá trị từ input
    return 0;
}
```

Python

```
# Cách 1: Đọc 2 số trên cùng 1 dòng, cách nhau bởi dấu cách
a, b = map(int, input().split())

# Cách 2: Đọc mỗi số trên 1 dòng riêng
a = int(input())
b = int(input())
```

Giải thích cách đọc input trong Python:

- `input()` đọc một dòng từ input dưới dạng xâu (string).
- `input().split()` tách xâu thành danh sách các phần tử (theo dấu cách).
- `map(int, ...)` chuyển từng phần tử sang số nguyên.
- `a, b = ...` gán các giá trị vào biến `a` và `b`.

Mẹo C++: Thêm dòng sau vào đầu `main()` để tăng tốc nhập/xuất:

```
ios_base::sync_with_stdio(false);
cin.tie(NULL);
```

4. Xuất dữ liệu (Output)

In kết quả ra standard output (stdout):

C++

```
int x = 42;
cout << x << endl; // In số, xuống dòng
cout << "Ket qua: " << x; // In kèm chữ
cout << x << " " << x * 2 << endl; // In nhiều giá trị
```

Python

```
x = 42
print(x) # In số, tự xuống dòng
print("Ket qua:", x) # In kèm chữ (tự thêm dấu cách)
print(x, x * 2) # In nhiều giá trị, cách bởi dấu cách
```

Quan trọng trên Online Judge:

- In đúng format theo yêu cầu đề bài (dấu cách, xuống dòng).
- Không in thêm dòng chữ thừa như "Nhập a: " — chỉ in kết quả.
- Thường cuối output cần có xuống dòng (`endl` hoặc `\n` trong C++, `print()` trong Python tự thêm).

5. Các phép toán số học

Đây là các phép toán cơ bản bạn sẽ dùng thường xuyên:

PHÉP TOÁN	C++	PYTHON	VÍ DỤ
Cộng	<code>a + b</code>	<code>a + b</code>	$5 + 3 = 8$
Trừ	<code>a - b</code>	<code>a - b</code>	$5 - 3 = 2$
Nhân	<code>a * b</code>	<code>a * b</code>	$5 * 3 = 15$
Chia (thực)	<code>(double)a / b</code>	<code>a / b</code>	$7 / 2 = 3.5$
Chia nguyên	<code>a / b</code>	<code>a // b</code>	$7 // 2 = 3$
Chia dư	<code>a % b</code>	<code>a % b</code>	$7 \% 2 = 1$

Ví dụ tổng hợp:

C++

```
#include <iostream>
using namespace std;
int main() {
    int a = 17, b = 5;
    cout << "Tong: " << a + b << endl; // 22
    cout << "Hieu: " << a - b << endl; // 12
    cout << "Tich: " << a * b << endl; // 85
    cout << "Chia nguyen: " << a / b << endl; // 3 (không phải 3.4!)
    cout << "Chia du: " << a % b << endl; // 2
    // Muốn chia thực, phải ép kiểu:
    cout << "Chia thuc: " << (double)a / b << endl; // 3.4
    return 0;
}
```

Python

```
a = 17
b = 5
print("Tong:", a + b) # 22
```

```
print("Hieu:", a - b) # 12
print("Tich:", a * b) # 85
print("Chia thuc:", a / b) # 3.4 (Python tự chia thực!)
print("Chia nguyên:", a // b) # 3
print("Chia dư:", a % b) # 2
```

Cảnh báo phép chia trong C++: Khi chia hai số nguyên trong C++, kết quả luôn là số nguyên (phần thập phân bị cắt bỏ). Nếu muốn kết quả thực, phải ép kiểu: `(double)a / b`.

6. Ví dụ hoàn chỉnh: Bài toán "Tổng hai số"

Đề bài: Cho hai số nguyên a và b . In ra tổng của chúng.

Input: Một dòng chứa hai số nguyên a và b .

Output: In ra tổng $a + b$.

C++

```
#include <iostream>
using namespace std;
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int a, b;
    cin >> a >> b;
    cout << a + b << endl;
    return 0;
}
```

Python

```
a, b = map(int, input().split())
print(a + b)
```

7. Xuất số thực với độ chính xác

Nhiều bài toán yêu cầu in số thực với số chữ số thập phân cố định. Ví dụ, in kết quả với 2 chữ số sau dấu chấm:

C++

```
#include <iostream>
#include <iomanip> // Cần cho fixed và setprecision
using namespace std;
int main() {
    double x = 3.14159;
```

```

    cout << fixed << setprecision(2) << x << endl;
    // Output: 3.14
    double y = 10.0 / 3.0;
    cout << fixed << setprecision(4) << y << endl;
    // Output: 3.3333
    return 0;
}

```

Python

```

x = 3.14159
print(f"{x:.2f}")
# Output: 3.14
y = 10 / 3
print(f"{y:.4f}")
# Output: 3.3333
# Cách khác dùng format():
print("{:.2f}".format(x))
# Output: 3.14

```

8. Ví dụ: Tính diện tích và chu vi hình chữ nhật

Đề bài: Cho chiều dài a và chiều rộng b của hình chữ nhật. In ra diện tích và chu vi, mỗi số trên một dòng.

C++

```

#include <iostream>
using namespace std;
int main() {
    int a, b;
    cin >> a >> b;
    cout << a * b << endl; // Diện tích
    cout << 2 * (a + b) << endl; // Chu vi
    return 0;
}

```

Python

```

a, b = map(int, input().split())
print(a * b) # Diện tích
print(2 * (a + b)) # Chu vi

```

9. Thứ tự ưu tiên phép toán

Giống như toán học, phép nhân/chia được tính trước phép cộng/trừ. Dùng ngoặc $()$ để thay đổi thứ tự:

ƯU TIÊN	PHÉP TOÁN	MÔ TẢ
1 (cao nhất)	()	Ngoặc
2	*, /, %	Nhân, chia, chia dư
3 (thấp nhất)	+, -	Cộng, trừ

Ví dụ:

- $2 + 3 * 4 = 2 + 12 = 14$ (không phải 20!)
- $(2 + 3) * 4 = 5 * 4 = 20$

10. Mẹo và lỗi thường gặp

Lỗi 1: Tràn số (Overflow) trong C++

Khi hai số int nhân nhau, kết quả có thể vượt quá giới hạn của int (2×10^9):

C++

```
int a = 100000, b = 100000;
// SAI: a * b = 10^10, vượt quá int!
int c = a * b; // Kết quả SAI (tràn số)
// ĐÚNG: Dùng long long
long long d = (long long)a * b; // Kết quả đúng: 10000000000
```

Python

```
# Python không bị tràn số - số nguyên có kích thước tùy ý!
a = 100000
b = 100000
c = a * b # 10000000000, không vấn đề gì
```

Lỗi 2: Chia nguyên bất ngờ trong C++

C++

```
// SAI: 7 / 2 = 3 (mất phần thập phân!)
int a = 7, b = 2;
cout << a / b; // In ra 3
// ĐÚNG: Ép kiểu sang double
cout << (double)a / b; // In ra 3.5
```

Lỗi 3: In thừa hoặc thiếu

- Không in "Nhập số: "hay "Kết quả là: "— online judge so khớp output chính xác.
- Kiểm tra kỹ: có cần xuống dòng cuối không, dấu cách giữa các số ra sao.

Lỗi 4: Quên đọc hết input

Nếu đề bảo input có 3 số mà bạn chỉ đọc 2, chương trình có thể cho kết quả sai.

Mẹo chung

- Luôn đọc kỹ đề bài trước khi code — chú ý giới hạn dữ liệu và format output.
- Test với ví dụ mẫu trước khi nộp.
- Nghĩ về trường hợp đặc biệt: số 0, số âm, số rất lớn.
- Trong C++, nên dùng `long long` nếu kết quả có thể lớn hơn 2×10^9 .